Chapter 7

Algorithm design and problem solving

IN THIS CHAPTER YOU WILL:

- learn about the use of, and stages in, the program development life cycle
- use decomposition to split a system into sub-systems
- create structure diagrams, flowcharts and pseudocode algorithms
- explain how a bubble sort and linear search works
- describe and produce algorithms that include finding the maximum, minimum and average values
- understand the need for validation and verification and write programs that use both
- identify appropriate test data for an algorithm
- complete a trace table for an algorithm
- learn how to check a program for errors and amend the program
- learn how to explain the purpose of an algorithm.

GETTING STARTED

Set up an obstacle course using chairs and other objects. Write down a set of instructions to guide a friend through the obstacle course, for example, stating how many steps to take, which way to turn, etc.

Give the instructions to a friend and ask them to follow the instructions safely (e.g. stop if they are about to hit something). Amend the instructions until your friend can successfully get through the obstacle course.

THE USE OF ALGORITHMS

Consider what is meant by an algorithm. If you search for algorithm then you will find answers such as a process, or a set of rules, that are followed. This does not mean they are limited to computers. You can find algorithms in all areas of life. You will have followed many of these without even considering that they are algorithms. For example, in mathematics you will have followed a series of steps to solve a problem.

Algorithms can take many forms, and use a variety of words or structures.

Computers follow algorithms. Programs are written as a series of instructions that the computer then follows. Algorithms for software might be millions of lines long, and they have to make use of specific instructions to tell the processor what to do.

A computer can only run machine code, that is binary. If you enter a command into the computer, this has to be converted into binary for the processor to execute (run) it.

Different programming languages have different focuses. Some different types are shown:

Declarative

This declares rules and you can ask it questions and it will give an answer based upon the rules. For example, here are some facts:

`person(ali)`	Ali is a person.
`person(jennifer)`	Jennifer is a person.
`person(tyler)`	Tyler is a person.
`friend(jennifer,tyler)`	Jennifer is friends with Tyler is a person.
`friend(tyler,ali)`	Tyler is friends with Ali.

From this you could ask who Jennifer is friends with, and it will return Tyler.

Object-oriented

This allows the user to create objects that have features (attributes) and processes (methods). For example, you could have a car as an object. Its features are colour, engine size, number of doors. Its processes are move forward, move backwards, turn.

Event driven

Some programs use buttons, text boxes and other items that the user can interact with. An event driven language has code that is only run when one of these items is clicked, or changed. The code is not run from start to finish, each item has its own code.

CONTINUED

Procedural

This has statements that are run in the order that they are written. It makes use of subroutines (procedures and functions) that can be called from other parts of the program. This is the type of language that you are most likely to start learning to program with.



Figure 7.1: A person reading programming statements

Discussion questions

1 What are the key features of an event driven language?

2 Why do you think there is a need for different types of programming language? Can you think of any types of problem that lend themselves to one type of language?

7.1 Program development life cycle

When a person, or organisation, creates a new computer program they will use a structured, organised plan of how to create the program. This is called the **program development life cycle**.

There are several varieties of life cycle, including cyclic, spiral, waterfall and rapid development. These all include the stages **analysis**, design, coding and testing.

KEY WORDS

program development life cycle: a series of structure step / activities that are followed to produce a system.

analysis: the first stage of the program development life cycle that involves investigating the problem.

7.2 Analysis

This first stage involves looking at the problem and the system that is needed. The problem is explored, and the requirements of the program are identified. **Decomposition** is used to split the problem into subproblems. Subproblems allow the developers to identify the requirements for each component part, and then tackle these separately in the next stage. Once you have decomposed the problem, each of the subproblems can be implemented as a subprogram.

7.3 Design

Once the requirements have been identified, the program designers can begin planning how the program will work in the **design** phase. This can include an overview of the program using a structure diagram, and the designing of algorithms using flowcharts and pseudocode.

Structure diagram

A **structure diagram** is developed by decomposing a program into its subprograms. The diagram has the name of the program at the top, and below this its subprograms. Each subprogram can be split down further as well if required.

Example 1

A calculator needs to allow the user to enter numbers and the symbol, calculate the result and output this.

The name of the program is at the top. The boxes below are ready for subprograms (Figure 7.2).

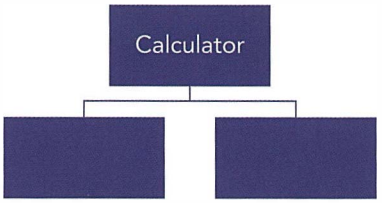


Figure 7.2: A structure diagram for a calculator

One way of splitting up a small program is to think of inputs, processes and outputs. A calculator has an input, processes and output, so these can be added to the diagram (Figure 7.3).

The subprograms can then be split further. The calculator needs two numbers and a symbol to be entered (e.g. 2 + 3). Add these boxes below input (Figure 7.4).

The **process** is dependent on the symbol entered. In this calculator, there is +, − , * and /. Each of these could be given a separate box (Figure 7.5).

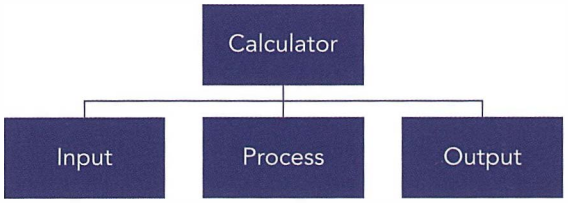


Figure 7.3: Input, process and output are added to the calculator structure diagram

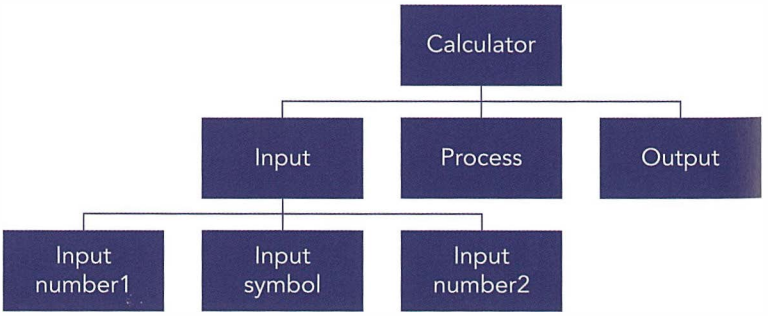


Figure 7.4: Expanding the input subprograms

KEY WORDS

decomposition: taking a system and splitting it into smaller sub-systems, which can in turn be split into smaller sub-systems.

design: the second stage of the program development life cycle, which involves decomposition of the problem and algorithms created ready for implementation.

structure diagram: a hierarchical diagram that shows the decomposition of a system.

process: an action that is performed, for example, `X ← X + 1.`

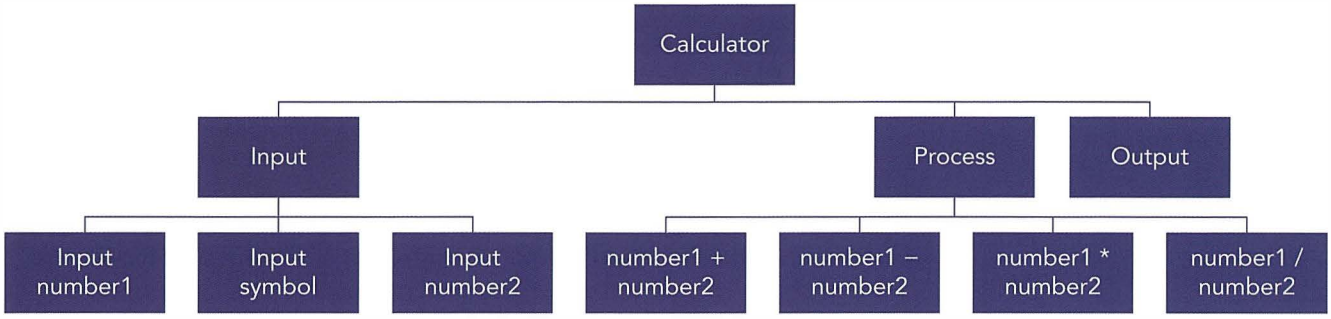


Figure 7.5: Expanding the process subprograms

Or the variable names from the input can be used (Figure 7.6):

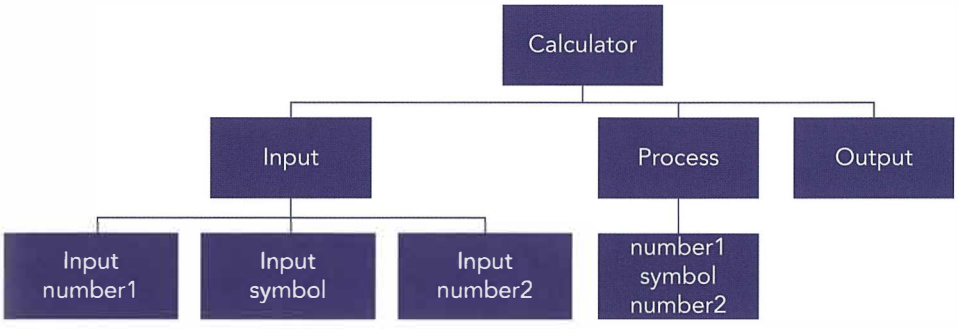


Figure 7.6: An alternative way of representing the algorithm shown in Figure 7.4

Both of these are valid. There are lots of ways you can **decompose** a problem, and everyone could do it slightly differently but the solution is still valid.

Finally, the output needs identifying (Figure 7.7):

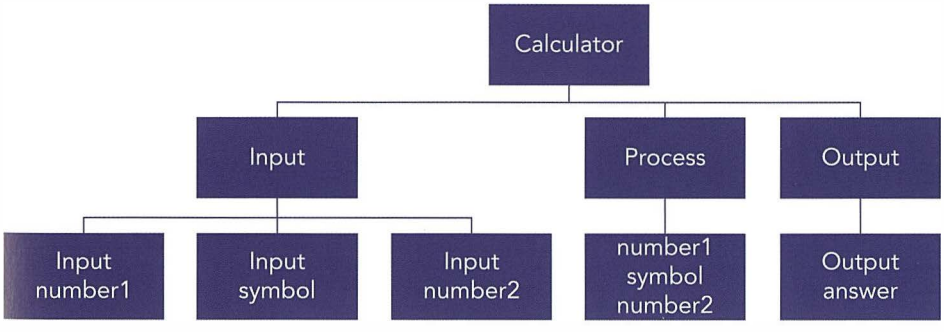


Figure 7.7: A finished structure diagram for the calculator

This program was fairly small, so it has been split into very precise instructions that can then be programmed. The next example is a larger problem.

A computerised version of chess is being developed. Two players can enter their names and then a new board will be displayed. The players take it in turns to select pieces to move and the positions to move them to. The program will check that the moves are valid and then move the pieces. It will remove any pieces that have been taken in each move, and check if anyone has reached checkmate.

KEY WORD

decompose: the action of performing decomposition; splitting a system into smaller sub-systems, which can in turn be split into smaller sub-systems.

Example 2

The name of the program is at the top (Figure 7.8).

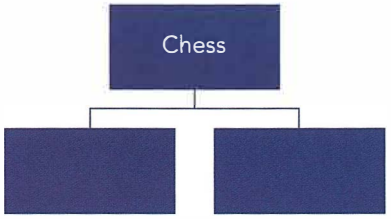


Figure 7.8: A structure diagram for a chess program

The program could be decomposed into inputs, processes and outputs, but this is a large program so it can be split down by different functions instead. The first part is that there are two players who need entering, this is given the name 'New players'. Then the game is set up ('New game'). The players can then move their pieces ('Make move'). The program will check if someone has checkmate ('Check won') (Figure 7.9):

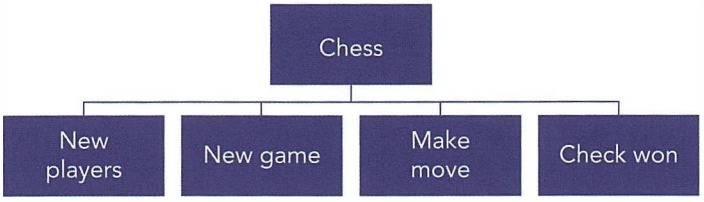


Figure 7.9: Subprograms for the chess program

These can then be split further. They are each given their own sub-diagram here so that they are easier to view (Figures 7.10–13):

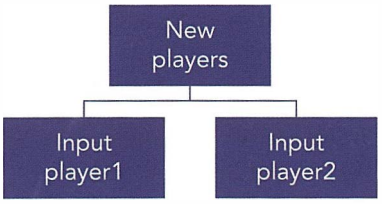


Figure 7.10: The New players subprogram

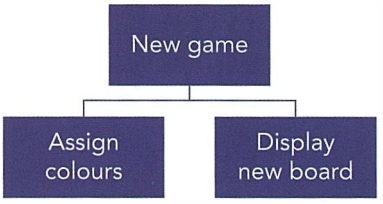


Figure 7.11: The New game subprogram

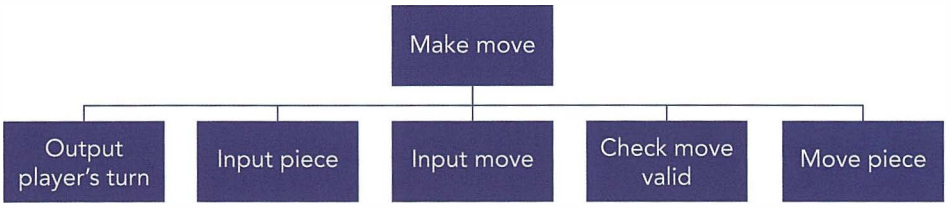


Figure 7.12: The Make move subprogram

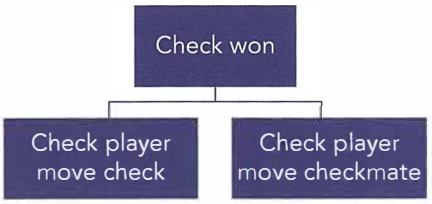


Figure 7.13: The Check won subprogram

ACTIVITY 7.1

Can you split any of these subprograms down further? In 'Check won' (Figure 7.13) what should happen if the result is checkmate? Should there be an output? What if there is check but not checkmate? Should this be output?

Peer Assessment

Compare your subsystems with a friend. Did you have the same answers? If so, are they both valid structure diagrams even though they are different?

Once you have decomposed your program into subprograms, you will need to design an algorithm for each part. This could be the level 1 (the first row below the top box) like in the calculator example. It could be for a lower level, like in the game of chess. Each box should become an individual algorithm. When you write your program, each of these individual algorithms might be a separate function or procedure (see Chapter 8, Section 12). Together, these individual algorithms should come together to create the whole system.

Questions

1. Describe the process of decomposition.
2. A program asks the user to enter two numbers. The program then adds the numbers together and outputs the total. Draw a structure diagram for the program.
3. A satellite navigation system needs to ask the user to input their destination. It then finds the route to the destination and outputs the instructions to the user. Draw a structure diagram for the system.
4. A login system asks the user to enter their username and password. It checks these are valid and either outputs that they are correct, or incorrect and prompts to re-enter. Draw a structure diagram for the system.

Flowcharts

A **flowchart** is a diagrammatic representation of an algorithm. Figure 7.14 shows some of the set of standard symbols.

KEY WORD

flowchart: a diagrammatic representation of an algorithm.

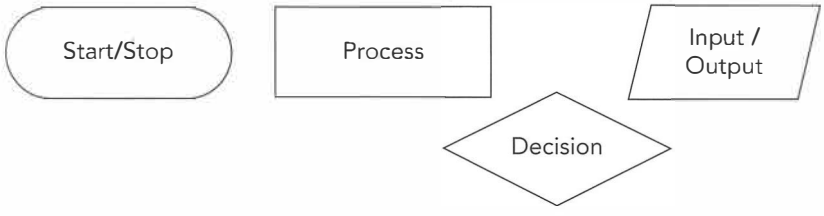
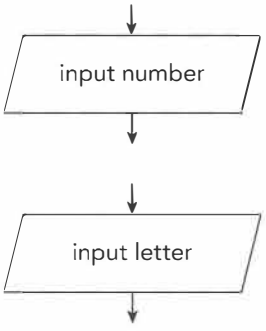
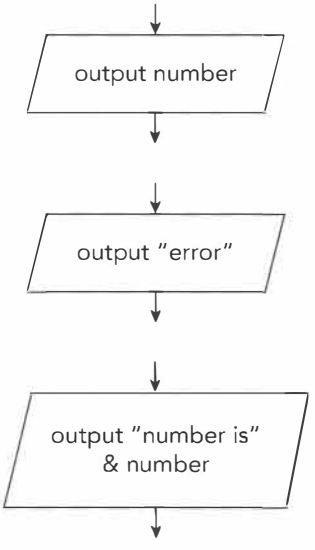


Figure 7.14: Flowchart symbols

A flowchart shows each step of a process, in the order that each step is performed. Each shape has one statement within it, for example, input number, or add 1 to x. The shapes are joined with arrows to show the direction of flow and the content inside each box can be written as words, or as pseudocode statements. Table 7.1 shows examples of how the flowchart symbols are used.

Shape	Description	Examples
Start/stop	A flowchart begins with a Start box, with one arrow from it. A flowchart ends with a Stop box, with one arrow going into it.	 **Figure 7.15:** Start/Stop
Input	A single input of a value with an identifier for that value. One arrow should go into the box, and one arrow should come out.	 **Figure 7.16:** Input
Output	An output of text in speech marks, and/or the identifier of a value	 **Figure 7.17:** Output

(continued)

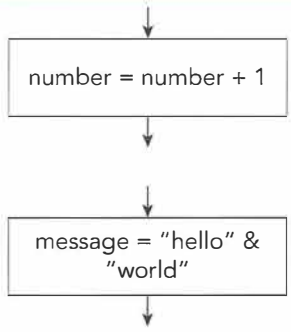
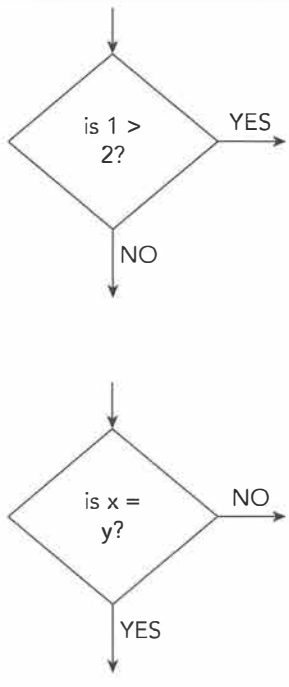
Shape	Description	Examples
Process	An action that is being performed, usually a mathematical calculation. One arrow goes into the box, and one arrow comes out of the box.	 **Figure 7.18:** Process
Decision	A question with two outcomes, yes or no (or true or false). The question is a comparison from: = equal to > greater than < less than >= greater than <= less than. You can have more than one condition by including AND or OR. See Chapter 8, Section 6 for the use of these operators. One arrow goes into the box. Two arrows come out, one labelled yes (or true) and the other no (or false).	 **Figure 7.19:** Decision

Table 7.1: Flowchart symbol examples

It is important that all arrows are complete, i.e. all boxes lead somewhere else so that you cannot get stuck in the flowchart with nowhere to go.

Example 1

Figure 7.20 shows a flowchart for a calculator.

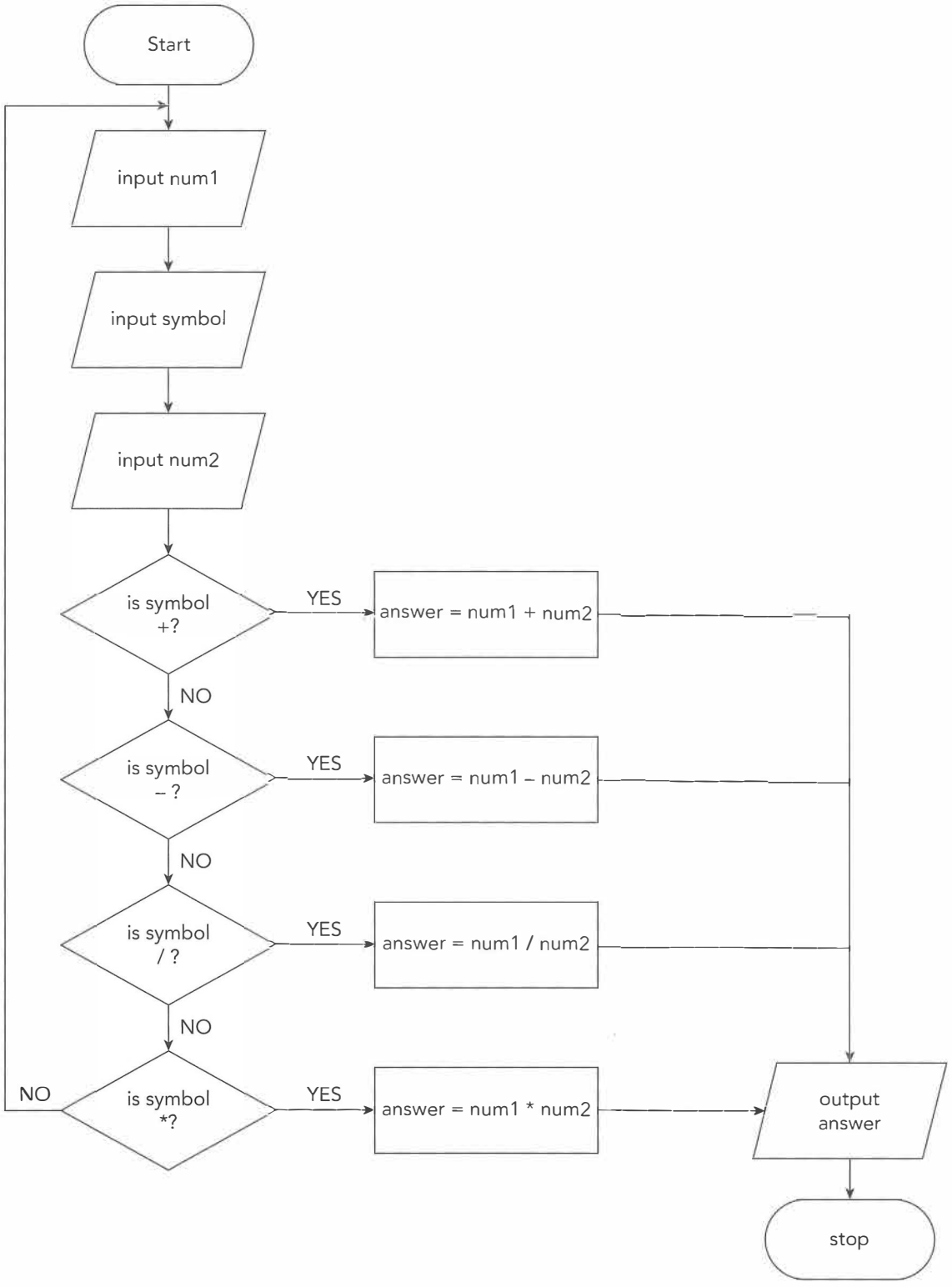


Figure 7.20: A flowchart for a calculator

Example 2

Figure 7.21 shows a flowchart for a password validation program.

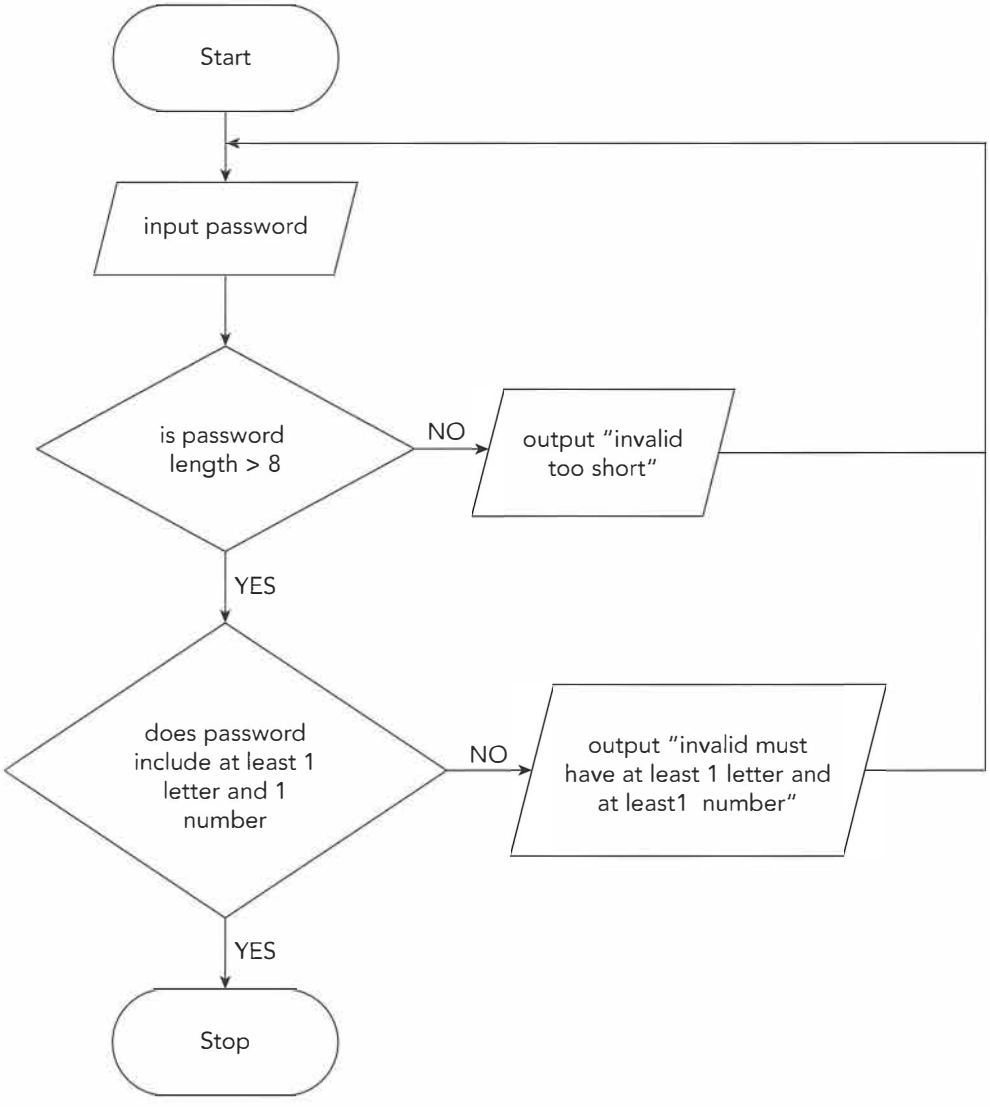


Figure 7.21: A flowchart for a password validation program

ACTIVITY 7.2

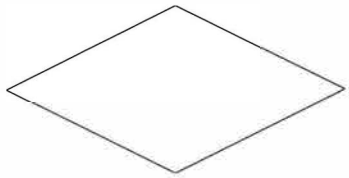
Flowcharts can be used for more than writing programs. Work in pairs to produce a flowchart for making a cup of tea, or coffee, or another drink.

Questions

5 Write the name of each flowchart symbol in the shape.

a 

b 

c 

d 

6 Draw the flowchart symbol for each statement:

a Start

b INPUT x

c INPUT y

d IS x > y?

e x = x + y

f Stop

7 Draw a flowchart to take three numbers as input, find and output the largest number.

7.4 Pseudocode

Pseudocode refers to any code that is not written on a computer to run. There is no one set pseudocode, because if there was then this would be code with a specific language. Instead it's a term for any readable code-like statements that all programmers will understand. This means that it uses keywords, and constructs such as IF statements, WHILE loops, etc. These constructs occur in almost all languages, so any programmer would be expected to read them and translate them into the actual programming language they are using.

KEY WORD

pseudocode: code-like statements that are used to design an algorithm but do not follow any specific language.

Example pseudocode:

```
INPUT Number1
IF Number1 < 10 THEN
    OUTPUT "too small"
ELSE
    OUTPUT "valid"
ENDIF
```

This uses the keywords INPUT, IF and OUTPUT. These can then be translated into whatever language the programmer is using. Here the pseudocode program is converted into Python, VB.NET and Java.

Python

```
number1 = input()
if number < 10:
    print("too small")
else:
    print("valid")
```

VB.NET

```
number1 = console.readline()
if number < 10 then
    console.writeline("too small")
else
    console.writeline("valid")
endif
```

Java

```
number = Integer.parseInt(scanner.nextLine());
if (number < 10){
    System.out.println("too small");
}else{
    System.out.println("valid");
}
```

There are many different valid pseudocode formats. Some will always use capitals for the commands, e.g. INPUT, IF, FOR. Some will use ← instead of an = for assignment. If you write your pseudocode in almost perfect 'code', i.e. you actually write Python code, because it is not to be run and you can have all sorts of syntax errors it is still pseudocode. What is important is that the program is split into steps that can be easily converted into a real program. Table 7.2 shows some examples of valid and invalid pseudocode.

Valid pseudocode	Invalid pseudocode
INPUT Value FOR X = 0 to 9 OUTPUT Value + X NEXT X	Input the value X Loop 10 times In each loop output the loop number added to the input value
num1 = input num2 = input if num1 > num2 then print(num1) else print(num2) Endif	input = num1 input = num2 if num1 is greater than num2 then output num1 if num1 is not greater than num2 then output num2

Table 7.2: Valid/invalid pseudocode examples

TIP

The syllabus includes information about the pseudocode conventions that will appear within examinations. The pseudocode conventions in this book don't always exactly follow the conventions you will see in your exams, but they are still valid. Remember that your syntax doesn't have to be perfect – it is your logic that is important.

PROGRAMMING TASK 7.1

A computer game stores a number for the user to guess. The user has up to 10 attempts to guess the number. After each turn the program outputs whether the number they guessed is smaller, larger, or equal to the number they are trying to guess. If they have 10 guesses without getting it correct, tell them what the answer is. If they get it correct, output the number of attempts it took them.

Getting started

1. Play the game with a partner. Discuss the different steps that you follow. Use this to Identify the inputs, processes and outputs required by this problem.
2. Recap the different components of a structure diagram and the purpose of decomposition.
3. Read an algorithm you have previously written that makes use of: input, output, selection and loops. Revisit the steps you identified in Task 1, which of the four areas is needed in each part.

Practice

1. Decompose the system into its subsystems by creating a structure diagram.
2. Draw a flowchart for the game.
3. Convert your flowchart into pseudocode.

Challenge

1. Is your code efficient? Are there any variables that are used that don't need to be? Can you reduce the number of lines of code?
2. Amend your program so that each time it runs, a random number is generated for the user to guess.

REFLECTION

After completing Programming Task 7.1:

How did you approach the programming task? Did you skip the getting started or did you jump straight to the practice?

Did that approach work? Will you consider the getting started next time?

Did you tackle the challenge yourself?

If not, would you revisit this when you are feeling more confident in this topic?

ACTIVITY 7.3

Find an example of a flowchart and turn it into a pseudocode algorithm.

Questions

8 Describe the purpose of pseudocode.

9 Tick one box in each row to identify if the statements are appropriate pseudocode statements, or not.

Statement	Valid	Invalid
`x = x + 1`		
`INPUT z`		
`IF y < 6 THEN`		
`add 3 to the value entered`		
`store the value entered in the variable newValue`		
`value1 = INPUT("Enter a number")`		
`IF number is more than 10`		
`WHILE number1 <> 0`		

10 Write a pseudocode algorithm to:
- take three values as input
- multiply the numbers together
- output the result.

11 Write a pseudocode algorithm to:
- take 10 numbers as input (using a loop)
- add together the values
- output the total with an appropriate message.

7.5 Coding

Once you have decomposed your problem into subproblems, and you have designed the algorithms using flowcharts and/or pseudocode then you can start writing the program in your chosen programming language. This is often referred to as **coding**.

This will also include some iterative testing. This is testing that is carried out while the program is being developed, for example, if you write the program code for one of the subproblems, then you will test it with a range of different data to make sure it is fully working before moving onto the next step.

7.6 Testing

When you have finished your program, you need to carry out **testing** to make sure it:

- fully works
- does not crash
- meets all requirements.

You will need to identify appropriate **test data** to use to test the program.
There are four types of test data:

- **Normal** – data that the program should accept.
- **Abnormal** – data that the program should not accept.
- **Extreme** – data that is at the edge of what is allowed.
- **Boundary** – data that is on the edge of being accepted and being rejected.

KEY WORDS

coding: the writing of a program using one or more programming languages.

testing: repeated use of a system to try all different possibilities to make sure the system is fully working and cannot be broken.

test data: the input data that is used to test a system.

normal test data: data that a program should accept.

abnormal test data: data that a program should not accept.

extreme test data: data that is on the edge of what is allowed.

boundary test data: data that is on the edge of being accepted, and data that is on the edge of not being accepted.

Example 1

A program allows the user to enter their age as a whole number between 10 and 100 years.

Test data type	Example data
Normal	Any number between 10 and 100 inclusive, e.g. 10, 20, 50, 100, etc.
Abnormal	Any data that is not a number between 10 and 100, e.g. 9, −1, 101, 200, "age", "30", 12.2.
Extreme	The largest number accepted – 100. The smallest number accepted – 10.
Boundary	Either side of the largest bound – 100 and 101. Either side of the smallest bound – 9 and 10.

Example 2

A user needs to enter a password at least 8 characters long.

Test data type	Example data
Normal	Any set of 8 or more characters, but they must be text (have speech marks), e.g. "abcdefgh" "12fghj567812".
Abnormal	Any set of characters with 7 or less characters, e.g. "1234567" , "seven".
Extreme	Any with 8 characters only, e.g. "12345678", "ajrkdhJK".
Boundary	Either side of the 8 character bound, 7 characters or 8 characters, e.g. "1234567", "12345678", "abcdefg", "iJHFD7hJ",

ACTIVITY 7.4

In pairs find a computer game to play. Anything as simple as a puzzle game, to a full 3D adventure game. Discuss how this game might have been tested. What data would be used as inputs?

Questions

12 Identify **three** different types of test data.

13 Why can you have the same test data as normal and extreme types of test data?

14 What is the difference between extreme and boundary test data?

15 A program asks the user to input a number between 1 and 10 (inclusive).

Identify test data for each of the test data types in the table:

Test type	Test data
Normal	
Abnormal	
Extreme	
Boundary	

7.7 Common algorithms

A **search algorithm** checks a set of data to identify whether a specific value exists in the data, or not. One example of this is a linear search.

A **sorting algorithm** takes a set of data and rearranges it to be in a specific order, e.g. in ascending alphabetical order. One example of this is a bubble sort.

KEY WORDS

search algorithm: a series of steps that searches for a specific value in a set of data.

sorting algorithm: a series of steps that will rearrange a set of data into an order, e.g. ascending numerical data.

COMPUTER SCIENCE IN CONTEXT

Sorting data and searching for data are activities that a very large proportion of programs will use at some point. For example, outputting a list of data may need sorting into order. A user may enter a username and the program needs to search for this in the set of valid usernames.

There are many different searching and sorting algorithms, each one has its benefits and drawbacks depending on the problem. This could be the number of data items it has to sort through, or whether the data is already in order to search for a value. It is up to the programmer to identify the algorithm that is most appropriate for their program to make sure it is as efficient as possible (both in the time it takes to run, and the amount of memory it needs to perform this task).

Linear search

In a program you might need to look for a specific value in a set of data. A **linear search** will check each item one at a time, starting with the first item and continuing until it either finds the item, or it checks the last value.

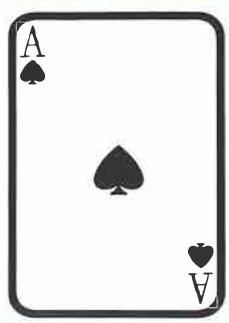
KEY WORD

linear search: a search algorithm that visits each item of data in turn to check whether it is the data being looked for.

ACTIVITY 7.5

Get a set of playing cards. Select 10 random cards and place them face down on the table in a row. Perform a linear search to see if the Ace of Spades is in the set.

1. Turn over the first card.
2. Is it the Ace of Spades?
3. If it is, then stop searching, you've found it.
4. If it isn't, turn over the next card. Repeat from 2.



Example 1

Search this set of data for the number 4.

5	3	9	4	2	1	8

Compare the first number (5) with the search value (4). They are not the same.

Compare the second number (3) with 4. They are not the same.

Compare the third number (9) with 4. They are not the same.

Compare the fourth number(4) with 4. They are the same. It has been found.

Example 2

Search this set of data for the character "#".

"a"	"v"	"!"

Compare the "a" with "#". They are not the same.

Compare the "v" with "#". They are not the same.

Compare the "!" with "#". They are not the same.

There are no more values, so "#" is not found.

Example 3

Searching an array (Chapter 8, Section 15 on arrays).

The array `dataArray` stores a set of data under one identifier. Each element in the array has an index.

Index	0	1	2	3
Data	1	5	6	8

Search the array for the number 6

Is `DataArray[0]` = 6? *No,* `DataArray[0]` = 1

Is `DataArray[1]` = 6? *No,* `DataArray[1]` = 5

Is `DataArray[2]` = 6? *Yes,* `DataArray[2]` = 6

6 was found in index 2.

This can now be written as an algorithm:

```
FOR ArrayIndex = 0 to LENGTH(Array) -1 //loop through each element
    //check if the current index is the data searched for
    IF DataArray[ArrayIndex] = SearchValue THEN
        //if it is, output the index where it is found
        OUTPUT "Found at " & ArrayIndex
    ELSE
        //if it not found, increment arrayIndex to check the next value
        ArrayIndex ← ArrayIndex + 1
    ENDIF
NEXT ArrayIndex
```

This algorithm is inefficient because if it finds the search value then it still continues searching. This is useful if you want to know whether the search value appears more than once.

You can make the algorithm more efficient by stopping as soon as the value is found, for example:

```
Found ← FALSE
ArrayIndex ← 0
//run while the value is not found, and you have not checked
//all elements
WHILE Found = FALSE AND ArrayIndex < LENGTH(Array) DO
    //check if the current index is the data searched for
    IF DataArray[ArrayIndex] = SearchValue THEN
        //if it is output where it was found
        OUTPUT "Found at " & ArrayIndex
        //set found to be true to stop the while loop running
        //again
        Found ← TRUE
    ELSE
        //if it is not found, increment ArrayIndex to the next value
        ArrayIndex ← ArrayIndex + 1
    ENDIF
ENDWHILE
```

TIP

Different programming languages use different characters to indicate the start of a comment (there is no closing character). For example, #, //, /* and '. You have to use one of these characters when writing pseudocode, but any one of these is acceptable.

Bubble sort

When writing a program you may need to sort a set of data into a specific order, for example, ascending numerical order or descending alphabetical order.

One method of sorting data is using a **bubble sort**.

KEY WORD

bubble sort: a sorting algorithm that moves through the list repeatedly swapping values in pairs.

A bubble sort takes the first 2 values; value 1 and 2, and compares them. If they are the wrong way around it swaps them. It then puts these back and takes values 2 and 3 and compares them. If they are the wrong way around it swaps them. This repeats until it has worked all the way through the list once. It then starts again with the first two values.

It is called a bubble sort because it acts like a bubble moving across all the elements to the end, then starting from the beginning.

There are **two** ways to tell the algorithm when to stop.

1. The algorithm has been through the entire list, the number of elements in the list −1. So if there are 10 elements, the algorithm runs 9 times. If there are 100 elements, the algorithm runs 99 times. If there are `XElements`, the algorithm runs `XElements - 1` times. This is the easiest way to program it, but it is not efficient. All elements might be in the correct order to begin with, but it is still going to run through the same number of times.
2. Either the algorithm has run through the list `XElements - 1` times, **or** it has run through the list, checking all of the elements, and it has **not** made any changes.

ACTIVITY 7.6

Get a set of playing cards. Select 10 random cards and place them face down on the table in a row. Perform a bubble sort to put the cards in ascending numerical order.

1. Turn over the first two cards.
2. If the 1st is larger than the 2nd, swap them. Otherwise, do nothing.
3. Turn the cards back over.
4. Turn over the 2nd and 3rd cards.
5. Repeat step 2.
6. Repeat turning over the cards and swapping until you have been through all 10 cards once.
7. Did you make any swaps? If so, start again from step 1. If not, stop because the cards should be in order.

A bubble sort can be written as a pseudocode algorithm. There are different versions of the algorithm. Here are two different versions.

Version 1

Stopping only when it has run array length −1 times.

```
//loop array length - 1 number of times
FOR NumberPasses ← 0 to LENGTH(DataArray) - 1
    //loop through each element in the array
    FOR Index ← 0 to LENGTH(DataArray) - 1
        //check if the data is in the correct order
        IF DataArray[Index] > DataArray[Index + 1] THEN
            //if not swap the items
            Temp ← DataArray[Index]
            DataArray[Index] ← DataArray[Index + 1]
            DataArray[Index + 1] ← Temp
        ENDIF
    NEXT Index
NEXT NumberPasses
```

Version 2

Stopping when there are no changes or when it has run array length − 1 times.

```
NumberPasses ← 0
//continue until one pass has no changes (changes = false)
//or it has looped array length - 1 times
WHILE Changes = FALSE or NumberPasses <= LENGTH(DataArray) - 1 DO
```

```
        //reset changes each time a new pass starts
        Changes ← FALSE
        //loop through each element in the array

        //check if the data is in the correct order
        FOR Index ← 0 to LENGTH(DataArray) - 1
            IF DataArray[Index] > DataArray[Index + 1] THEN
                //if not swap the items
                Temp ← DataArray[Index]
                DataArray[Index] ← DataArray[Index + 1]
                DataArray[Index + 1] ← Temp
                Changes ← TRUE
        ENDIF
        NEXT Index
    ENDWHILE
```

SKILLS FOCUS 7.1

BUBBLE SORT OF NUMERICAL DATA

When you are performing a bubble sort, first identify the order required, e.g. ascending or descending. Then identify the data item being sorted, e.g. is it a number, a letter, a set of words, etc.

For example: sort the numbers 6 2 5 1 into ascending numerical order.

There are 4 numbers that need sorted into *ascending numerical* order. This means you keep on sorting until either:

a you have passed through all numbers 3 times

or

b you have passed through all the numbers once without making any changes.

To help guide your working it's often useful to draw a table for each of the items.

Draw a table with the four values:

6	2	5	1

Pass 1. Work through the data items, comparing each pair in turn. If the data is in the wrong order, swap them.

Data				Instructions
6	2	5	1	
6	**2**	5	1	Compare the first two values.
2	**6**	5	1	They are in the wrong order so swap them.
2	**6**	**5**	1	Compare values 2 and 3.
2	**5**	**6**	1	They are in the wrong order so swap them.
2	5	**6**	**1**	Compare values 3 and 4.
2	5	**1**	**6**	They are in the wrong order so swap them.

CONTINUED

Now you have completed 1 pass, check if it should stop!

Compare to the two criteria:

a you have passed through all numbers 3 times **[False, only 1 pass has been made]**

or

b you have passed through all the numbers once without making any changes **[False, there has been at least one change]**.

Pass 2. Work through the data items, comparing each pair in turn. If the data is in the wrong order swap them.

Data				Instructions
2	5	1	6	
2	**5**	1	6	Compare the first two values.
2	**5**	**1**	6	They are in the correct order. Compare values 2 and 3.
2	**1**	**5**	6	They are in the wrong order. Swap them.
2	1	**5**	**6**	Compare values 3 and 4. They are in the correct order.

Should it stop?

Compare to the two criteria:

a you have passed through all numbers 3 times **[False, only 2 passes have been made]**

or

b you have passed through all the numbers once without making any changes **[False, there has been at least one change]**.

Pass 3. Work through the data items, comparing each pair in turn. If the data is in the wrong order swap them.

Data				Instructions
2	1	5	6	
2	**1**	5	6	Compare the first two values.
1	**2**	5	6	They are in the wrong order. Swap them.
1	**2**	**5**	6	Compare values 2 and 3.
1	2	**5**	**6**	They are in the correct order. Compare values 3 and 4.
1	2	**5**	**6**	They are in the correct order.

Should it stop?

Compare to the two criteria:

a you have passed through all numbers 3 times **[True, 3 passes have been made so stop]**.

CONTINUED

Questions

1. How many swaps were made in this bubble sort?
2. Why did it stop after 3 passes?
3. How would you change this process to put the data into descending numerical order?

SKILLS FOCUS 7.2

BUBBLE SORT WITH CHARACTERS INTO DESCENDING ORDER

Put the characters "z" "c" "f" "a" "h" into descending alphabetical order.

First identify the data and method. This is string data, and into descending alphabetical order.

There are five characters, so keep on sorting until either:

a you have passed through all characters 4 times

or

b you have passed through all the numbers once without making any changes.

Draw a table with the given characters:

z	c	f	a	h

Pass 1. Work through each of the data items once. If the pair is in the incorrect order, swap them.

z	c	f	a	h
z	**f**	**c**	a	h
z	f	c	**h**	**a**

Should it stop? It has not run 4 times. It has made changes. Repeat.

Pass 2.

z	f	c	h	a
z	f	**h**	**c**	a

Should it stop? It has not run 4 times, and it made a change. Repeat.

Pass 3.

z	f	h	c	a
z	**h**	**f**	c	a

Should it stop? It has not run 4 times. It has made 1 change. Repeat.

CONTINUED

Pass 4.

z	f	h	c	a

Should it stop? It has run 4 times – stop.

Questions

1 How many comparisons were made that did not result in swaps?

2 How would you change this algorithm to work with strings instead of multiple characters, e.g. 'zoo' 'farm' 'cat' 'horse' 'animal'?

PROGRAMMING TASK 7.2

A program asks the user to enter a set of 20 numbers. The user can then enter a number for the program to search for in the set of data. The program will output if it is found, or not. The program should then put the data into ascending order and output the newly arranged data.

Getting started

1 Practise the linear search process using a set of cards. Lay them out and search them in turn to identify if a specific one is present.

2 Practise the bubble sort process using a set of cards. Lay them out and put them into the correct order following the bubble sort instructions.

3 Recap the programming skills; input, output, selection and iteration. Focus on how to loop 20 times to enter 20 numbers.

4 Read the linear search and bubble sort algorithms with a partner. Follow each instruction in the algorithm.

Practice

1 Identify the inputs, processes and outputs for the problem.

2 Create a structure diagram for the problem.

3 Write a pseudocode algorithm to take in the set of 20 numbers from the user.

4 Amend the algorithm to allow the user to input a number to search for, and then the code to perform a linear search on the data and output the result.

5 Amend the program to perform a bubble sort on the data, and output the arranged program.

Challenge

1 Move the linear search into its own subroutine (procedure or function) so it can be called from anywhere in the main program.

2 Move the bubble sort into its own subroutine and call it from the program.

Questions

16 Perform a linear search for the number 6 in the following data:

10 5 6 8 1 2

17 Perform a linear search for the letter "f" in the following data:

b u l p f a g h

18 Perform a bubble sort to put the following data into ascending numerical order:

1 5 6 2 9 0 1

19 Perform a bubble sort to put the following data into descending numerical order:

60 2 40 52 3 5

20 Explain the difference in efficiency between the two different bubble sort algorithms.

Totalling

Totalling is adding a group of values together to give a total.

KEY WORD

totalling: statements in a program that add together a set of data to produce the total.

For example, a person buys items at a supermarket. The items cost: $0.50, $1.00, $1.00, $15.50, $21.30 The total is 0.5 + 1 + 1 + 15.5 + 21.3 = $39.30

To do this in a program you need to:

- Initialise a variable to store the total, e.g. `Total ← 0`.
- Add the value to the current contents of the total, e.g. `Total ← Total + Value`.

Example 1

Totalling the 10 values input by the user:

```
Total ← 0  //Initialise the total variable to 0
FOR X ← 0 TO 9
    Total ← Total + INPUT("Enter a value") //Add the input to the total
NEXT X
OUTPUT Total
```

Example 2

Totalling the values in an array named `dataArray`:

```
TotalValue ← 0  //Initialise the total variable to 0
FOR Count ← 0 TO LENGTH(DataArray) - 1
    TotalValue ← TotalValue + DataArray[Count] //Add the input to the total
NEXT Count
OUTPUT Total
```

PROGRAMMING TASK 7.3

A program is needed to ask the user to input the price of products they have bought. Calculate and output the total cost they spent.

Getting started

1 What are the steps required to find the total? For example, initialising the total to 0 will be the first step.

CONTINUED

2 You will need to work out how to continue asking for the price of products until they do not have any more. Have a look at some of the previous algorithms in this chapter that perform similar actions to this. What did they use? How did they limit the number that could be entered?

Practice

1 Create a structure diagram for the problem.

2 Write an algorithm to ask users to repeatedly input the price of products they have bought until there are no more to enter.

3 Amend the algorithm to total the cost of all the products and output this at the end of the algorithm.

Challenge

1 Amend the algorithm to also count how many items they entered. You can look ahead to the next section or work out how to count them yourself.

2 Now you have the total and the quantity, amend your program to work out the average cost.

Counting

KEY WORD

counting: statements in a program that record how many of something there are.

Counting is working out how many of something there are. For example, counting how many fingers you have or counting how many items you bought when you went shopping.

To do this in a program you need to:

- Initialise a variable to start the count, e.g. `CountItems ← 0`.
- Add 1 to the count variable each time, e.g. `Count ← Count + 1`.

Example 1

Count the number of items the user enters:

```
Count ← 0 //initialise the count variable
Number ← 0
WHILE Number <> -1 DO // loop until the user enters -2
    Number ← INPUT("Enter a number. Enter -1 to stop")
    Count ← Count + 1 //add 1 to count each time a number is input
ENDWHILE
```

Example 2

Count how many numbers in an array are below 10, and how many are more than 20:

```
CountBelow10 ← 0 //initialise both counters to 0
CountAbove20 ← 0
FOR X ← 0 to LENGTH(NumberArray) //loop through each array element
```

```
    //if the number is less than 10
    IF NumberArray[x] < 10 THEN
       CountBelow10 ← CountBelow10 + 1 //increment the counter
    //if the number is greater than 20
    ELSEIF NumberArray[x] > 20 THEN
       CountAbove20 ← CountAbove20 + 1 //increment the counter
    ENDIF
NEXT X
```

PROGRAMMING TASK 7.4

A program needs to ask the user to input the weight of parcels being posted. It needs to count how many parcels are less than 1 kg, how many are between 1 kg and 2 kg, and how many are above 2 kg.

Getting started

1. Recap how to count in an algorithm. What are the different pieces of code you need? For example, initialise the count to start at 0.
2. You will need to use selection to compare the inputs with the weights. Recap the use of selection and the format, including the differences between the operators >, >=, < and <=.

Practice

1. Draw a structure diagram for the problem.
2. Write an algorithm to read in the weights of the parcels until the user has finished.
3. Amend the algorithm to count and output how many parcels are below 1 kg.
4. Amend the algorithm to also count and output how many parcels are between 1 and 2 kg.
5. Amend the algorithm to also count and output how many parcels are more than 2 kg.

Challenge

The user would like to enter the limits that they are counting themselves when the program starts. For example, counting how many parcels are less than 1.5 kg instead of 1 kg. Amend the program to make this change.

Finding the minimum

The **minimum** value is the smallest value within a set. For example, the marks for a class are entered and the lowest mark is the minimum.

KEY WORD

minimum: the smallest item in a set of data.

To do this in a program you need to:

- initialise a minimum variable to be a large value, beyond those that will be entered, e.g. `Minimum ← 9999`.
- Compare each value to the minimum variable, e.g. `IF Number < Minimum`.

- If it is, it replaces the minimum value, e.g. `minimum ← number`.
  This means the new value is the minimum, so the next time a value is compared, it is checking it with the new minimum value.

Example 1

Find the smallest number input by a user:

```
Minimum ← 9999 //initialise minimum to large value
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number ← INPUT("Enter a number or 0 to stop")
    IF Number < Minimum THEN // check if the number entered is smaller than
                             //the current minimum
        Minimum ← Number //if true then make minimum because the number
    ENDIF
ENDWHILE
```

Example 2

Find the smallest number in an array of data:

```
Minimum ← 999999
FOR X ← 0 TO LENGTH(NumberArray)
    IF NumberArray[X] < minimum THEN
        Minimum ← NumberArray[X]
    ENDIF
NEXT X
```

Finding the maximum

KEY WORD

maximum: the largest item in a set of data.

The **maximum** value is the largest value within a set. For example, the marks for a class are entered and the highest mark is the maximum.

To do this in a program you need to:

- Initialise a maximum variable to be a small value, beyond those that will be entered, e.g. `Maximum ← -9999`.
- Compare each value to the maximum variable, e.g. `IF Number > Maximum`.
- If it is, it replaces the maximum value, e.g. `Maximum ← Number`.
  This means the new value is the maximum, so the next time a value is compared, it is checking it with the new maximum value.

Example 1

Find the largest number input by a user:

```
Maximum ← -9999 //initialise maximum to small value
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number = INPUT("Enter a number or 0 to stop")
    IF Number > Maximum THEN // check if the number entered is larger than
                             //the current maximum
        Maximum ← Number //if true then make maximum because the number
    ENDIF
ENDWHILE
```

Example 2

Find the largest number in an array of data:

```
Maximum ← -1 // initialise the maximum value to a small number
FOR X ← 0 TO LENGTH(NumberArray)
    //check if the number entered is larger than the current maximum
    IF NumberArray[X] > Maximum THEN
        Maximum ← NumberArray[X] //if true store the value in maximum
    ENDIF
NEXT X
```

Finding the average

KEY WORD

average: the mean of a set of values, the total divided by the quantity.

The **average** here is referring to the mean. This is the total of all the values added together, then divided by how many numbers there are.

For example, if the data is 1, 3, 5, 8, 4, 2, 6, 9 ,

the average = (1 + 3 + 5 + 8 + 4 + 4 + 6 + 9) / 8 = 40 / 8 = 5

To do this in a program you need to use the count and total from earlier in this section. The average is then calculated with the formulae `Total / Count`.

Example 1

Find the average of the numbers input by a user:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
Number ← 1
WHILE Number >= 1 DO //loop until the user enters 0
    Number ← INPUT("Enter a number or 0 to stop")
    Total ← Total + Number //add the number entered to the total
    Count ← Count + 1 //add 1 to the count
ENDWHILE
```

Example 2

Find the average of the numbers in an array of data:

```
Count ← 0 //initialise the count to 0
Total ← 0 //initialise the total to 0
FOR X ← 0 TO LENGTH(NumberArray)
    Total ← Total + NumberArray[X] //add value to total
    Count ← Count + 1 //add 1 to count
NEXT X
```

PROGRAMMING TASK 7.5

A teacher needs a system to calculate the lowest, highest and average mark their 30 students got in a test out of 100.

Getting started

1 Identify the code required to work out the minimum and maximum values in a set of data.

CONTINUED

2 Recap how to calculate an average of a set of numbers.

3 30 numbers will need entering. Recap how to loop 30 times.

Practice

1 Write an algorithm to allow the teacher to enter the marks for all 30 students.

2 Amend the program to calculate and output the smallest (minimum) mark.

3 Amend the program to calculate and output the largest (maximum) mark.

4 Amend the program to calculate and output the average mark.

Challenge

Extend the program to count how many got below the average, and how many got above the average number of marks.

Extend the program to work out how many students got each level. 80 or more = distinction. 60 or more = merit. 40 or more = pass. Below 40 = fail.

Questions

21 Explain the difference between totalling and counting.

22 A program allows numbers to be entered between 20 and 200, and finds the smallest and largest entered. Identify appropriate values to initialise the variables `Smallest` and `Largest`.

23 Write an algorithm to input 40 numbers and output the total and average.

24 Write an algorithm to input 100 numbers and output the smallest and largest.

Validation on input

Validation is the checking of data that is input to make sure it is reasonable, and/or within set bounds. For example, making sure that a number is entered for an age or limiting the range of numbers that can be entered.

KEY WORD

validation: the checking of data to make sure it is reasonable and within set bounds.

ACTIVITY 7.7

Have you ever entered data to be told that it is invalid? What were you trying to enter, and why was it incorrect? Visit some websites that need you to enter some data (but stay safe and make sure it is not personal data).
What restrictions do they have? Are there some spaces that you need to enter data in? Do some of them give you a set structure to fill in, for example, a date as _ _ / _ _ / _ _ _ _?
Are these all controlled with validation rules?

There are different types of validation.

Range check

A **range check** assesses whether data is within one or two bounds. For example, an age must be between 0 and 100. A date must be before today's date.

Range checks can be programmed using selection to produce an error, or within a loop that keeps asking you enter a value until it is valid.

KEY WORD

range check: a type of validation that makes sure data is between the minimum and maximum.

Example 1

Using selection:

This algorithm will check whether a number is higher than 1 and less than 10.

```
Number ← INPUT("Enter a number between 1 and 10")
IF Number < 1 OR Number > 10 THEN
    OUTPUT("Invalid")
ELSE
    OUTPUT("Valid")
ENDIF
```

Example 2

Using a loop:

This algorithm will check if a number is higher than 1 and less than 10. It will continually ask for a value to be input until the number is valid.

```
Number ← INPUT("Enter a number between 1 and 10")
WHILE Number < 1 OR Number > 10 DO
    OUTPUT("Invalid please try again")
    Number ← INPUT()
ENDWHILE
```

Length check

A **length check** will check the number of characters that are present. This could in a string, for example, the length of "hello world" is 11, the space is also a character. It could be in a variable, for example, in this example, 8 will be output:

KEY WORD

length check: a type of validation that checks the number of characters is within a set limit.

```
TheData ← "123 ABC!"
OUTPUT(LENGTH(TheData))
```

The length of a piece of data can be found using a variety of commands depending on the language you are using. The following are all valid and there are many more:

```
theData.length()
LENGTH(theData)
LEN(theData)
theData.len
```

A length check can be programmed using selection or iteration.

Example 1

Using selection:

This algorithm will check if the data input has 10 or less characters to be invalid, otherwise (10 or more) it is valid.

```
Data ← INPUT()
IF LENGTH(Data) < 10 THEN
    OUTPUT("Invalid")
ELSE
    OUTPUT("Valid")
ENDIF
```

Example 2

Using iteration:

This algorithm will also check the length but will continually ask for this to be input until it is 10 or more characters long.

```
Data ← Input()
WHILE LENGTH(Data) < 10 DO
    OUTPUT("Invalid, please try again")
    Data ← INPUT()
ENDWHILE
```

Type check

Data can be in different forms, including an integer (whole number), real (decimal number), string (any characters), Boolean (true or false). There are two ways of writing a **type check**:

1. Use a function such as `.GetDataType()` to return the data type, that you can then compare to the one you want.
2. Use a function such as `.IsInteger()` to return True if it is an integer and False otherwise.

KEY WORD

type check: a type of validation that checks data is the correct data type.

Example 1

Using selection:

This algorithm will check if the data entered is an integer value to be valid.

```
INPUT Data
//check if the data entered is an Integer
IF Data.GetDataType() <> Integer THEN //if it is not an integer
    OUTPUT ("Invalid")
ENDIF
```

Example 2

Using iteration:

This algorithm will continually ask for the data to be input until it is a string value.

```
INPUT Data
//loop while the data entered is not a string
WHILE Data.IsString = FALSE DO
    OUTPUT("Invalid please try again")
    INPUT Data
ENDWHILE
```

Presence check

A **presence check** makes sure that some data has been entered. Programming languages can make use of the value `null`, or for a string value an empty string represented by `""`.

KEY WORD

presence check: a type of validation that makes sure data has been entered.

Example 1

Using selection:

This algorithm outputs Invalid if there is no data entered.

```
INPUT Data
IF Data = NULL THEN  //check if the data entered is null
   OUTPUT("Invalid")
ENDIF
```

Example 2

Using iteration:

This algorithm continually takes an input while there is no data entered.

```
INPUT Data
WHILE Data = "" DO //loop while there is nothing in data
   OUTPUT("Invalid please try again")
   INPUT Data
ENDWHILE
```

Format check

Some data may need to be entered in a specific way, for example, a date must be: _ _ / _ _ / _ _ _ _, where each space is a number – a **format check** makes sure the data is entered this way. An ID number may need to be 1 number followed by 3 characters.

KEY WORD

format check: a type of validation that makes sure the data meets a specific order, e.g. 1 number, then 3 letters.

Example 1

Using selection:

An ID number entered needs to be 1 number followed by 2 characters. This algorithm checks the characters and outputs whether it meets the required format.

```
INPUT IdNumber
//check if the first character is a number, and characters 2
//and 3 are strings
IF(SUBSTRING(IdNumber, 0, 1).IsNumeric = TRUE AND
   SUBSTRING(IsNumber, 1, 2) = TRUE) THEN
   OUTPUT("Valid")
ELSE
   OUTPUT("Invalid")
ENDIF
```

Note that `SUBSTRING(String, X, Y)` starts at character x in the string, and returns y number of characters.

Example 2

Using a loop:

This algorithm checks whether an input is in the format two numbers, /, two numbers, /, four numbers.

```
INPUT Date
//loop while the data is not in the correct format
WHILE (SUBSTRING(Date, 0, 2).IsNumeric = FALSE OR //2 numbers
       SUBSTRING(Date, 2, 1) <> "/" OR // 1st slash
       SUBSTRING(Date, 3, 2).IsNumeric = FALSE OR //2 numbers
       SUBSTRING(Date, 5, 1) <> "/" OR //2nd slash
       SUBSTRING(Date, 6, 4).IsNumeric = FALSE) DO // 4 numbers, year
   OUTPUT("Invalid")
   INPUT Date
ENDWHILE
OUTPUT("Valid")
```

Check digit

A **check digit** is calculated from a set of numbers, and is input with the numbers. When the data has been input, the calculation is performed on the data again and the result is compared with the final number (the check digit).

KEY WORD

check digit: a type of error detection method that is used for data entry. A calculation is performed on the data entered to create a value. Check digit values are compared to see if the data entered is correct.

Example

This algorithm calculates the check digit using the method 5 digit number input. 9999, 4 digits used to calculate 5th check digit, and compares it to the input value.

```
INPUT Code
//extract 1st digit
Digit1 ← Code DIV 1000
Code ← Code - (1000 * Digit1)
//extract 2nd digit
digit2 ← Code DIV 100
Code ← Code - (100 * Digit1)
//extract 3rd digit
Digit3 ← Code DIV 10
Code ← Code - (10 * Digit1)
//extract 4th digit
Digit4 ← DIV(Code, 1)
CheckDigit ← Code
//calculate check digit from data entered
//multiply by position and add together
Total ← (Digit1 * 4) + (Digit2 * 3) + (Digit2 * 2) + Digit3
NewCheckDigit ← MOD(Total, 11) //find Mod 11 of total
NewCheckDigit ← 11 - NewCheckDigit //subtract result from newCheckDigit
//check if the calculated check digit is the same as the last digit in the //code
IF NewCheckDigit = CheckDigit THEN
   OUTPUT("Correct data entry")
ELSE
   OUTPUT("Incorrect data entry")
ENDIF
```

You can find out more about check digits in Chapter 2.

Verification of data

When entering data from another source, for example, a paper copy, it is important to make sure you have entered exactly what was written originally. This is called **verification**, a check that you have copied the data accurately.

There is a difference between entering data accurately, and entering the correct data. Verification only checks you have copied it accurately, the actual data may still be incorrect if the original was incorrect.

Two forms of verification are a visual check and double entry.

A **visual check** is where you compare the data entered to the original. For example, reading each line from a paper copy and checking it is identical on the computer.

A **double entry check** is where the same data is entered twice, usually by different people. The computer will then check whether there are any differences in what they each entered.

KEY WORDS

verification: checking that data is entered accurately, that it is the same as the original.

visual check: comparing the data entered with the original side-by-side.

double entry check: two different people enter the same data which are then compared.

COMPUTER SCIENCE IN CONTEXT

Have you ever had to enter data into a program twice? You have most likely done this when entering your email address, or a password. Think about why it was important that your email and password were entered accurately. What could have happened if they were not?

PROGRAMMING TASK 7.6

A user needs to enter a series of data that each require validation. The table shows the data and the requirements:

Data	Validation requirements
Username	Minimum length of 5 characters.
Date of birth	In the format NN/NN/NNNN, e.g. 01/01/2020.
Type of character	Limited to: "Elf", "Fairy", "Gnome", "Magician".
Starting strength	A number between 1 and 5 inclusive.
Starting health	10 minus the starting strength input.

Getting started

For each of the validation requirements, decide on which type of validation you are going to use. Try and choose a different one for each value.

Recap the algorithms for each type, write an example beside each of the ones you have chosen.

CONTINUED

Practice

1 Write an algorithm to take as input the five values.

2 Amend the program to include the validation for each item of data.

3 Amend your program to output, for each data item, whether their data is valid or invalid.

Challenge

Extend the program to repeatedly ask for each item of data until they enter one that is valid. Then the next piece of data should be input.

Could any of the items of data have more than one validation rule used? Find at least one piece of data that can have two types of validation and implement it in your program.

ACTIVITY 7.8

Work in pairs to identify how many times you have come across validation or verification in a program or website. Compare your answers with another pair. How common are validation and verification? Are there any methods not identified here, for example, what is two-step verification?

Questions

25 What is validation?

26 Give **three** examples of validation checks.

27 What is the purpose of verification?

28 Identify which validation rule this algorithm uses:

```
INPUT Num
IF Num > 0 AND Num < 100 THEN
    OUTPUT "Valid"
ELSE
    OUTPUT "Invalid"
ENDIF
```

29 Identify which validation rule this algorithm uses:

```
REPEAT
    INPUT Value
UNTIL Value <> NULL AND Value <> ""
```

30 Write a validation routine to input a word and check that it is more than 10 characters long.

31 Write a validation routine to input a number and check that it is an integer value.

7.8 Trace tables

A **trace table** is a structure to help you follow an algorithm, for example, to find an error, or to work out what the algorithm does.

Each statement in an algorithm is run manually, and the values that are written to variables are written in the table in the order the changes are made.

Each column in the table is designated to one variable. There can also be a column for any outputs to be written in. User prompts are also entered in the trace table, these can be entered in a separate column or as an output (because they will be output). For example, x = input("Enter a number"), the text "Enter a number" is a user prompt and will need to be added to the trace table.

Specific inputs can be given to test different parts of the algorithm.

KEY WORD

trace table: a structure to complete when walking through an algorithm manually, where the values that change are written in each row.

SKILLS FOCUS 7.3

TRACE TABLE WITH SELECTION

To create a trace table you need to identify all of the variables in an algorithm. Each variable should have its own column in the trace table. There should also be an output column if the program will output any data or strings.

For example, trace the following algorithm with the input values 1 and 3.

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

There are **two** variables and there is an output (each variable has a column, and the output has a column).

number1	number2	OUTPUT

You then complete this table by reading through the algorithm one line at a time. It is important not to jump ahead and guess what the algorithm will do. Read one line and perform that action.

Run line 1 of the code (input values 1, 3):

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

CONTINUED

number1	number2	OUTPUT
1		

The first line inputs the first value given, 1.

Run line 2 of the code (input values 1, 3):

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

number1	number2	OUTPUT
1	3	

The second line inputs the second value given, 3.

Run the IF statement:

```
INPUT Number1
INPUT Number2
IF Number1 > Number2 THEN
    OUTPUT(Number1)
ELSE
    OUTPUT(Number2)
ENDIF
```

Use the values in the table. number1 is 1, number2 is 3.

IF 1 > 3 This is FALSE so run the ELSE

OUTPUT the value in number2

number1	number2	OUTPUT
1	3	3

ACTIVITY 7.9

Run the algorithm in Skills Focus 7.3 with the following input data:

a 10, 2

b 9, 9

c 20, 30

Peer Assessment

Compare your answers from running the algorithm with the different values with a partner. Did you get the same result? If not, work out which is correct by running the algorithms again.

SKILLS FOCUS 7.4

TRACE TABLE WITH LOOPS

When you are following an algorithm that includes a loop (iteration), some of the code will be repeated. To follow the code run one line at a time as before, and when you get to the end of the loop, go back to the start of the loop.

For example: trace the following algorithm with the input values 4, 3, 2, 1, 0.

```
01 Count ← 0
02 InputValue ← 1
03 Total ← 0
04 WHILE InputValue > 0 DO
05     INPUT InputValue
06     Total ← Total + InputValue
07     Count ← Count + 1
08 ENDWHILE
09 OUTPUT "The total is ", Total
10 OUTPUT "There were ", Count, " Numbers"
```

Start by creating the trace table with each of the variables and the output.
There are 3 variables in this algorithm:

Count	InputValue	Total	OUTPUT

1 Run the first 3 lines of code. These are the initialisations, lines 01, 02 and 03.

```
01 Count ← 0
02 InputValue ← 1
03 Total ← 0
```

Count	InputValue	Total	OUTPUT
0	1	0	

2 Run line 04.

```
04 WHILE InputValue > 0 DO
```

LOOP is 1 so this is TRUE meaning that the code inside the loop is run

Run line 05 with the first input value of 4.

```
05     INPUT InputValue
```

Count	InputValue	Total	OUTPUT
0	1	0	
	4		

When a variable has a new value, it is written below the previous one.

CONTINUED

3 Run lines 06 and 07.

```
06      Total ← Total + InputValue        Total ← 0 + 4
07      Count ← Count + 1                 Count ← 0 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	

4 Line 08:

```
08  ENDWHILE
```

sends you back to line 04.

```
04  WHILE InputValue > 0 DO
```

`InputValue` is 4, so while 4 > 0 is TRUE to run line 05 with the second input value of 3.

```
05      INPUT InputValue
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
	3		

5 Lines 06 and 07 run.

```
06      Total ← Total + InputValue    Total ← 4 + 3
07      Count ← Count + 1             Count ← 1 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	

6 Line `08  ENDWHILE` returns to line 04.

```
04  WHILE InputValue > 0 DO     WHILE 3 > 0 is TRUE. Run lines 05, 06 and 07.
05      INPUT InputValue                    InputValue ← 2
06      Total ← Total + InputValue          Total ← 7 + 2
07      Count ← Count + 1                   Count ← 2 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	

7 Line 08 again returns to line 04. WHILE 2 > 0 is TRUE. Run lines 05, 06 and 07.

```
05      INPUT InputValue                    InputValue ← 1
06      Total ← Total + InputValue          Total ← 9 + 1
07      Count ← Count + 1                   Count ← 3 + 1
```

CONTINUED

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	

8 Line 08 returns to line 04 again. WHILE 1 > 0 is TRUE. Run lines 05, 06 and 07.

```
05      INPUT InputValue                    InputValue ← 0
06      Total ← Total + InputValue          Total ← 10 + 1
07      Count ← Count + 1                   Count ← 4 + 1
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	
5	0	10	

9 Line 08 returns to line 04 again. WHILE 0 > 0 is FALSE this time, so the loop does not run.

Jump to line 09 run line 09 and 10.

```
09 OUTPUT "The total is ", Total
10 OUTPUT "There were ", Count, " numbers"
```

count	inputValue	total	OUTPUT
0	1	0	
1	4	4	
2	3	7	
3	2	9	
4	1	10	
5	0	10	The total is 10
			There were 5 numbers

Be careful with your outputs. They must match the output in the code. For example, '*The total is 10*' is correct, but '*total = 10*' is incorrect.

Questions

1. What do you record in a trace table?
2. When do you change the value in a column?

ACTIVITY 7.10

Trace the algorithm in Skills Focus 7.4 with the following inputs:

a 1, 6, 8, 2, 0

b 1, −1, 2

c 10, 20, 30, −5

Self Assessment

Trace the algorithms a second time and see whether you got the same result. If not, where did you go wrong?

7.9 Finding the purpose of an algorithm

A trace table can help you work out the purpose of an algorithm. By following an algorithm step-by-step you can see what happens to the values, what changes are made, and when these are made. From this you can determine the purpose of the algorithm.

SKILLS FOCUS 7.5

FINDING THE PURPOSE WITH A TRACE TABLE

Describe the processes in this algorithm and state its purpose.

```
INPUT Value1
INPUT Value2
INPUT Value3
IF Value1 > Value2 AND Value1 > Value3 THEN
   OUTPUT(Value1)
ELSEIF Value2 > Value1 AND Value2 > Value3 THEN
   OUTPUT(Value2)
ELSE
   OUTPUT(Value3)
ENDIF
OUTPUT(Value1 + Value2 + Value3)
```

1 First test the algorithm with a set of data that you come up with (if they are not provided for you). For example, test this algorithm using 1, 2 then 3.

2 Create a trace table by following the algorithm with the data input.

Value1	Value2	Value3	OUTPUT
1	2	3	3
			6

3 Repeat the process with a different set of data, for example, this time using 10, 5, 1.

4 Complete the trace table.

CONTINUED

Value1	Value2	Value3	OUTPUT
10	5	1	10
			16

Keep on repeating this process until you can identify the pattern.

In this case it is outputting the largest number input and then outputting the total of the 3 numbers input.

Questions

1 Two sets of data were used to run this algorithm. If you had to choose a third set what would you choose?

2 How do you know when to stop testing the algorithm with different data?

3 Describe the processes in this algorithm and state its purpose using a trace table.

```
Quantity ← 0
Total ← 0
Value ← 0
Small ← 999
WHILE Quantity < 5 DO
    INPUT Value
    IF Value >= 10 AND Value <= 100 THEN
        Quantity ← Quantity + 1
        Total ← Total + Value
        IF Value < Small THEN
            Small ← Value
        ENDIF
    ENDIF
ENDWHILE
OUTPUT("small = ", Small)
OUTPUT(Total / Quantity)
```

quantity	Total	value	small	OUTPUT

TIP

Copy the trace table and test the algorithm in Q3 with some sample data, e.g. 1, 100, 50, 10, 33, 3, 42.

SKILLS FOCUS 7.6

DESCRIBING AN ALGORITHM

When you are describing an algorithm you need to do more than repeat the line of code using English statements. For example, in the following code, for line 02 don't just say, 'it is a for loop from y is 0 to x.' This is stating what each part of the code is, but it does not describe the algorithm.

```
01 INPUT X
02 FOR Y ← 0 TO X
03     OUTPUT Y
04 NEXT Y
```

You can do a trace table to help you work out the purpose of the algorithm before describing it if that helps you. This algorithm has three parts: line 01, line 02 and line 03. These are summarised below:

- Takes a number as input. (line 01)
- Loops from 0 to the number that has been input. (line 02)
- Outputs the values from 0 to the number. (line 03)

From this you can develop a description of the algorithm, for example:

The algorithm takes a number from the user, and then outputs all of the numbers from 0 to the number input.

Questions

1 What methods can you use to work out the purpose of an algorithm?

2 Why should your description explain more than what each part of the code does?

3 Describe the purpose of this algorithm:

```
01 X ← -1
02 Count ← 0
03 WHILE Count < 10 DO
04     INPUT Y
05     IF Y > X THEN
06         X = Y
07     ENDIF
08     Count ← Count + 1
09 ENDWHILE
10 OUTPUT(X)
```

7.10 Finding errors in a program and correcting the errors

To find an error in an algorithm you need to know the purpose of the algorithm. You can then test the algorithm with different data, completing trace tables if needed, to find out what the algorithm actually does. Once you know the difference between what it does and should do, you can start to identify the errors and the changes you need to make.

COMPUTER SCIENCE IN CONTEXT

Computer programmers will come across errors all the time. These could be ones that they have made, or that they are testing or fixing for other people. There are lots of ways that they can find the errors, but many of them will use the two methods used here; using a trace table, and dry running by reading the code. There are other options often available to them as well, for example, features within the Integrated Development Environment (IDE) that they use to write the program. These features allow the programmer to see the values of variables and the outputs at each stage, without having to trace them manually.

SKILLS FOCUS 7.7

USING A TRACE TABLE TO FIND ERRORS IN A PROGRAM

Using a trace table will allow you to follow each step in the algorithm and by comparing this to the intended outcome you can find where it goes wrong, and then work out how to correct it.

For example, this program should take 5 numbers as input and output the total of all the numbers.

```
01 Total ← 1
02 Quantity ← 1
03 WHILE Quantity < 5 DO
04     INPUT Number
05     Total ← Number
06     Quantity ← Quantity + 1
07 ENDWHILE
08 OUTPUT(Total)
```

First dry run the algorithm. If not provided, you will need to identify some test data to use.

This algorithm should take 5 numbers as input, so you could use 1, 2, 3, 4, 5.

Then work out what the result of the algorithm should be. It outputs the total, so it should output: $1+2+3+4+5=15$

CONTINUED

Now draw a trace table for the algorithm (revisit the section on trace tables if needed) and then run each step of the algorithm, writing the values in the table.

Total	Quantity	Number	OUTPUT
1	1		
1	2	1	
2	3	2	
3	4	3	
4	5	4	

Now look at the results and where it has gone wrong:

1. The total is overwritten each time, it should be added to.
2. Total starts with 1 but this will add another 1 to the total, so it should be 0.
3. Only four numbers can be input, it should be five.

Let's take each of the problems one at a time to solve them.

1. The total should not be overwritten. Line 05 overwrites the total. This should be `Total ← Total + Number`.
2. The total should not start at 1, it should be 0. Line 01 writes 1 to total. This line should be `Total ← 0`.
3. Only 4 numbers are input, it should be 5. There are two ways this can be changed, both are valid. Either Line 02 can be `Quantity ← 0` or line 03 can be `WHILE Quantity < 6`.

SKILLS FOCUS 7.8

FINDING ERRORS BY READING THE CODE

You may not always need to use a trace table, or you may not want to. There are many different ways of finding the errors. In this example you will look for each of the requirements in the algorithm to determine whether it has been met.

The following algorithm should take 3 numbers as input, and output the largest value and the average value.

```
01 Total ← 0
02 Largest ← 999
03 FOR X ← 0 TO 2
04     INPUT Data
05     IF Data < 999 THEN
06         Large ← Data
07     ENDIF
08     Total ← Total + Data
09 NEXT X
10 OUTPUT Largest
11 Average ← Total / 3
12 OUTPUT Average
```

CONTINUED

Re-read the description: take 3 numbers as input, and output the largest value and the average value.

This can be split into three sections:

1 Take 3 numbers as input.
2 Output the largest value.
3 Output the average value.

The two outputs need to be calculated before being output, so they can be added to the requirements:

1 Take 3 numbers as input.
2 Calculate the largest value.
3 Output the largest value.
4 Calculate the average value.
5 Output the average value.

Now check the first requirement:

Does it take 3 numbers as input?

The for loop goes from 0 to 2, so it will run 0, 1, 2 = 3 times. Inside the loop it inputs a value each time. **This is correct**.

The second requirement:

Calculate the largest value.

To find the largest it needs to:

a Initialise a largest variable with a low number.

This is **not** met. Line 02 largest is set to 999 but this should be a very small value, e.g.

```
Largest ← -1
```

b It should check if the data input is larger than the data in largest.

This is **not** met. Line 05 checks if data is less than 999. It should be larger.

```
IF Data > Largest THEN
```

c Line 06 replaces the value in `Large` with data, but the identifier is **incorrect**, the variable is named largest.

```
Largest ← Data
```

The third requirement:

Output the largest value.

Line 10 outputs the largest value, **this is correct.**

CONTINUED

The fourth requirement:

Calculate the average value.

The average is calculated by totalling the inputs and then dividing by the number of inputs.

Line 08 totals the data entered, but it uses the **wrong identifier**, it should be data not DATA.

```
Total ← Total + Data
```

Line 11 works out the average **correctly**.

The fifth requirement:

Output the average value.

Line 12 outputs the average **correctly**.

In this case the errors and corrections have been identified along the way. You can also combine this with using test data to check both the original algorithm and to check that your corrections are accurate.

Questions

1 Complete the trace table for the following algorithm with the input data: 20 50 60 25 35.

```
Total ← 0
Average ← 0
Counter ← 0
WHILE Counter <= 5 DO
    INPUT Mark
    Total ← Total + Mark
    Counter ← Counter + 1
ENDWHILE
Average ← Total / Counter
OUTPUT "The total is ", Total
OUTPUT "The average is ", Average
```

Total	Average	Counter	Mark	OUTPUT

CONTINUED

2 **a** Complete the trace table for the following algorithm using the input data: 7.

```
INPUT Number
WHILE Number >= 0 DO
    OUTPUT Number
    Number ← Number - 1
ENDWHILE
```

Number	OUTPUT

b State the purpose of the algorithm.

3 Explain the purpose of the following algorithm:

```
Valid ← 0
Invalid ← 0
Continue ← "Y"
WHILE Continue = "Y" DO
    INPUT Value
    IF Value >= 10 AND Value <= 20 THEN
        Valid ← Valid + 1
    ELSE
        Invalid ← Invalid + 1
    ENDIF
    OUTPUT "Continue?"
    INPUT continue
ENDWHILE
OUTPUT "valid = ", Valid
OUTPUT "invalid = ", Invalid
```

SUMMARY

Programs are developed by following a program development life cycle.
The program development life cycle is made up of; analysis, design, coding and testing.
In the analysis stage the problem is identified and decomposed.
In the design stage the algorithms are planned.
In the coding stage the program is written.
In the testing stage the program is tested.
Decomposition is the splitting of a system into sub-systems.
Decomposition can be shown using a structure diagram.
Algorithms can be planned using flowcharts and pseudocode.
A linear search checks each item of data in turn.
A bubble sort compares the items in pairs repeatedly.
Totalling is adding together the values.
Counting is adding 1 to the count for each item.
To find the minimum value, a variable needs to be initialised with a large value.
To find the maximum value, a variable needs to be initialised with a small value.
Validation is checking data entered is reasonable and within bounds.
Validation checks include; range check, length check, type check, presence check, format check and check digit.
Verification checks that data has been input accurately.
Verification checks include visual check and double entry check.
Four types of test data are normal, abnormal, extreme and boundary.
A trace table is used to dry run an algorithm. It can be used to find the purpose of an algorithm and to find errors in the algorithm.

EXAM-STYLE QUESTIONS

1 **Define** the term algorithm. [1]

2 One stage of the program development life cycle is the analysis.
Identify and **describe** *two* other stages. [4]

3 Describe the purpose of decomposition. [2]

4 A computer game allows a user to load a saved game or to start a new game. The game then loads the puzzle for that level. The user controls a character by using the arrow keys on the keyboard. The left arrow moves the character left, the up makes the character jump, right moves the character right, down makes the character lie down.

a Draw a structure diagram for the game. [4]

COMMAND WORDS

define: give precise meaning.

identify: name / select / recognise

describe: state the points of a topic / give characteristics and main features.

CONTINUED

b Write an algorithm using a flowchart to represent the input of a key from a user, and then move the character. **[6]**

c When the user starts a new game, they have to input a name that is made of at least 8 letters. This is validated using a length check.

i Identify *one other* appropriate validation routine that could be used to validate the name. **[1]**

ii Write, using pseudocode, the length check validation for the name. **[4]**

[Total: 15]

5 Complete the trace table for the following algorithm, with the input data:

blue	5	Y	blue	1	Y	yellow	3	Y	white	5	x

```
Blue ← 10
White ← 5
Yellow ← 2
Continue ← "Y"
WHILE Continue = "Y" DO
    INPUT Colour
    INPUT Quantity
    IF Colour = "Blue" AND Quantity <= Blue THEN
        Blue ← Blue - Quantity
        OUTPUT "Blue left = ", Blue
    ELSEIF Colour = "White" AND Quantity <= White
        White ← White - Quantity
        OUTPUT "White left = ", White
    ELSEIF Colour = "Yellow" AND Quantity <= Yellow
        OUTPUT "Yellow left = ", Yellow
    ELSE
        OUTPUT "Try again"
    ENDIF
    OUTPUT "Continue?"
    INPUT Continue
ENDWHILE
```

Blue	White	Yellow	Continue	Colour	Quantity	OUTPUT

[5]

CONTINUED

6 **Show** the stages of a linear search, when looking for the number 100 in the following data:

10 20 8 6 5 3 1 **[3]**

7 Show the stages of a bubble sort to put the following data into ascending alphabetical order:

apple	pear	banana	pineapple	orange	peach

[5]

8 The following algorithm should:

- Ask the user to input a number 20 times.
- Count and output how many numbers are less than 10, equal to 10, and more than 10.
- Calculate and output the average number entered.

```
01 TotalLess ← 0
02 Equal ←  0
03 TotalMore ← 0
04 Total ← 0
05 FOR X ← 0 TO 20
06     INPUT Number
07     IF Number <= 10 THEN
08         TotalLess ← TotalLess + 1
09     ELSEIF Number <> 10 THEN
10         Equal ← Equal + 1
11     ELSE
12         TotalMore ← TotalMore + 1
13     ENDIF
14     Total ← Total + Number
15 NEXT X
16 OUTPUT "Total less = ", TotalLess
17 OUTPUT "Total equal = ", Equal
18 OUTPUT "Total more = ", TotalMore
19 OUTPUT "Average = ", Total / 20
```

The algorithm has *four* errors in it.

Identify the line number of each error and write the corrected line for each. **[4]**

COMMAND WORD

show (that): provide structured evidence that leads to a given result.

SELF-EVALUATION CHECKLIST

After studying this chapter, think about how confident you are with the different topics.
This will help you to see any gaps in your knowledge and help you to learn more effectively.

You might find it helpful to rate how confident you are for each of these statements when you are revising.
You should revisit any topics that you rated 'Needs more work' or 'Getting there'.

I can...	See topic	Needs more work	Getting there	Confident to move on
identify the stages in the program development life cycle.	7.1			
use decomposition to split a system into sub-systems.	7.2			
create structure diagrams, flowcharts and pseudocode algorithms.	7.3			
explain how a bubble sort and linear search works.	7.7			
describe and produce algorithms that include finding the maximum, minimum and average values.	7.7			
understand the need for validation and verification and write programs that make use of each.	7.7			
identify appropriate test data for an algorithm.	7.6			
complete a trace table for an algorithm.	7.8			
check a program for errors and amend the program.	7.10			
explain the purpose of an algorithm.	7.9			